

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“КАЗАНСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им. А.Н.
ТУПОЛЕВА-КАИ”**

**Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной безопасности**

**Отчет по лабораторной работе №1
По теме
“Основы OpenMP: параллельные области, распределение работ,
синхронизация потоков”
По дисциплине
“Аппаратные средства телекоммуникационных систем”**

**Выполнил студент
Группы 4404
Сайдаков Л.И.**

Казань, 2026

Цель работы: Изучение основ параллельного программирования для систем с общей памятью: параллельные области и идентификация потоков, распределение итераций цикла между потоками (типы расписания `schedule`), примитивы синхронизации потоков. В ходе работы выполнены задачи 1–3 методических указаний «Параллельное программирование на OpenMP». Работа выполняется на языке Go, поэтому конструкции OpenMP моделируются стандартными средствами языка: горутины и `runtime.GOMAXPROCS` соответствуют команде потоков, `sync.WaitGroup` — неявному барьеру в конце параллельной области, `sync.Mutex` — критической секции, `sync/atomic` — атомарным операциям, каналы — средствам обмена и синхронизации

Задания:

Задача 1. Написать программу, где каждый поток печатает свой идентификатор, количество потоков всего и строку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Задача 2. Написать программу, в которой объявлен массив из 16000 элементов, инициализированный так, что значение элемента равно его порядковому номеру. Создать результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: $b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0$. Запустить программу с 8 потоками при различных типах распределения работ.

Задача 3. Модифицировать задачу 1 так, чтобы потоки печатали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения; необходимо реализовать и продемонстрировать как можно больше из них.

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу; каждый запуск, о котором говорится в задаче (разное число потоков, разные типы распределения, разные версии

программы), подтверждается скриншотом с видимой командой запуска и полным выводом.

Ход работы.

Задача 1.

Программа считывает из командной строки число потоков N , ограничивает уровень параллелизма вызовом `runtime.GOMAXPROCS(N)` и создаёт N горутин (аналог команды потоков OpenMP). Каждая горутина печатает свой идентификатор, общее число потоков и строку «Hello World». Главная горутина ожидает завершения всех рабочих горутин с помощью `sync.WaitGroup`, что соответствует неявному барьеру на выходе из параллельной области. Мьютекс `sync.Mutex` используется только для того, чтобы строки разных потоков не перемешивались посимвольно; порядок вывода он не задаёт

Команды запуска:

```
cd lab1/task1
dotnet run -c Release -- 8
```

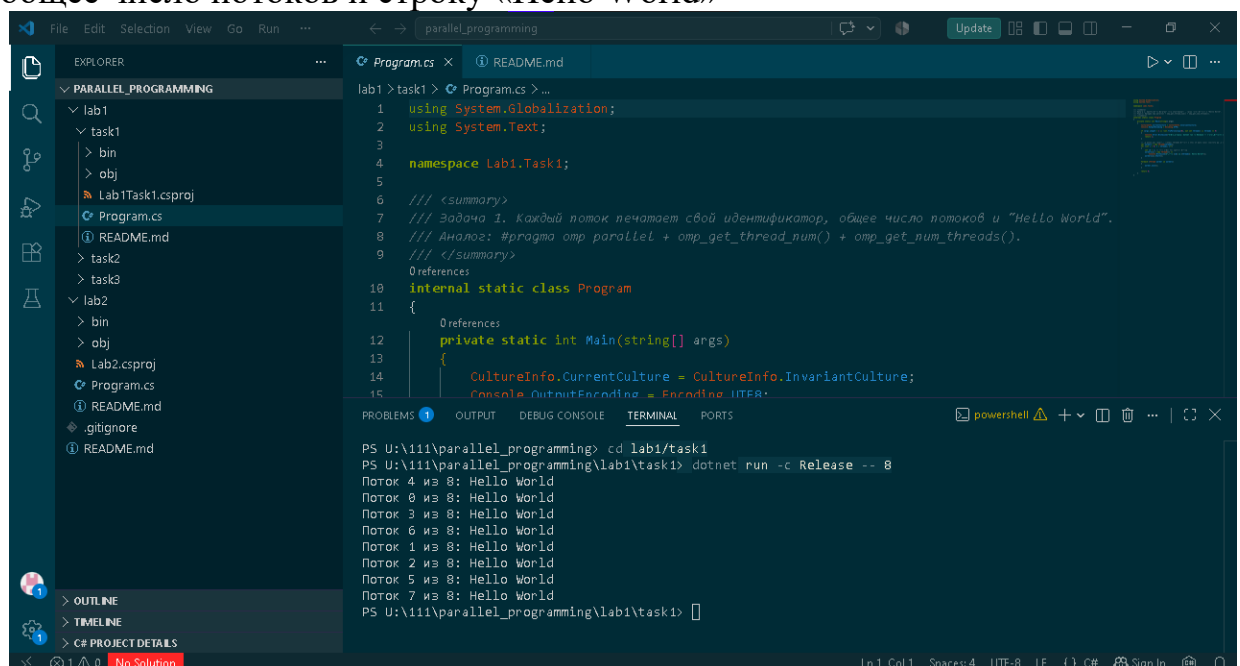
Ответ на вопрос

Нет, вывод не всегда идентичен. Потоки стартуют почти одновременно, а порядок их выполнения определяет планировщик операционной системы, поэтому порядок строк от запуска к запуску меняется. Каждая строка при этом выводится целиком.

Реализация

В цикле создаются N объектов `Thread`, каждый печатает свой `id` и `N`. Главный поток дожидается всех через `Join()`.

На рисунке 1 показан первый запуск программы с 8 потоками. Видна команда запуска и полный вывод: каждый из 8 потоков напечатал свой идентификатор, общее число потоков и строку «Hello World»



```
1 using System.Globalization;
2 using System.Text;
3
4 namespace Lab1.Task1;
5
6 /// <summary>
7 /// Задача 1. Каждый поток печатает свой идентификатор, общее число потоков и "Hello World".
8 /// Аналог: #pragma omp parallel + omp_get_thread_num() + omp_get_num_threads().
9 /// </summary>
10
11 internal static class Program
12 {
13     private static int Main(string[] args)
14     {
15         CultureInfo.CurrentCulture = CultureInfo.InvariantCulture;
16         Console.OutputEncoding = Encoding.UTF8;
17     }
18 }
```

```
PS U:\111\parallel_programming> cd lab1/task1
PS U:\111\parallel_programming\lab1\task1> dotnet run -c Release -- 8
Поток 4 из 8: Hello World
Поток 0 из 8: Hello World
Поток 3 из 8: Hello World
Поток 6 из 8: Hello World
Поток 1 из 8: Hello World
Поток 2 из 8: Hello World
Поток 5 из 8: Hello World
Поток 7 из 8: Hello World
PS U:\111\parallel_programming\lab1\task1>
```

Рис. 1 – запуск с 8 потоками

Задача 2.

Программа создаёт массив a из N элементов ($N = 16000$ передаётся через командную строку), где $a[i] = i$, и вычисляет результирующий массив b по формуле $b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0$ для внутренних индексов $i = 1 \dots N-2$. Крайние элементы $b[0]$ и $b[N-1]$ в условии не определены и копируются из исходного массива. Обработка внутренних элементов распределяется между 8 потоками в соответствии с выбранным типом расписания, аналогичным условию `schedule` в OpenMP:

- `static` без блока: диапазон итераций делится между потоками непрерывными равными частями;
 - `static` с блоком `chunk`: блоки фиксированного размера раздаются потокам циклически;
 - `dynamic`: потоки берут блоки из общей очереди (защищена мьютексом, аналог критической секции), размер блока по умолчанию 1;
 - `guided`: размер выдаваемого блока уменьшается по мере обработки массива, но не меньше минимального `chunk`;
 - `runtime`: тип расписания и размер блока читаются из переменной окружения
- Гонки данных отсутствуют: каждая итерация записывает только в свой элемент $b[i]$, массив a не изменяется. Поэтому результирующий массив и контрольная сумма не зависят от типа расписания (при $a[i] = i$ получаем $b[i] = i$, сумма равна $N \cdot (N-1) / 2 = 127992000$, программа выводит ОК: true). Различаются лишь статистика распределения итераций и блоков по потокам и время выполнения.

На рисунке 2 показан запуск с расписанием `static` без задания блока: 15998 внутренних итераций разделены между 8 потоками непрерывными участками (6 потоков по 2000 итераций, 2 потока по 1999). Контрольная сумма совпала с ожидаемой (ОК: true).

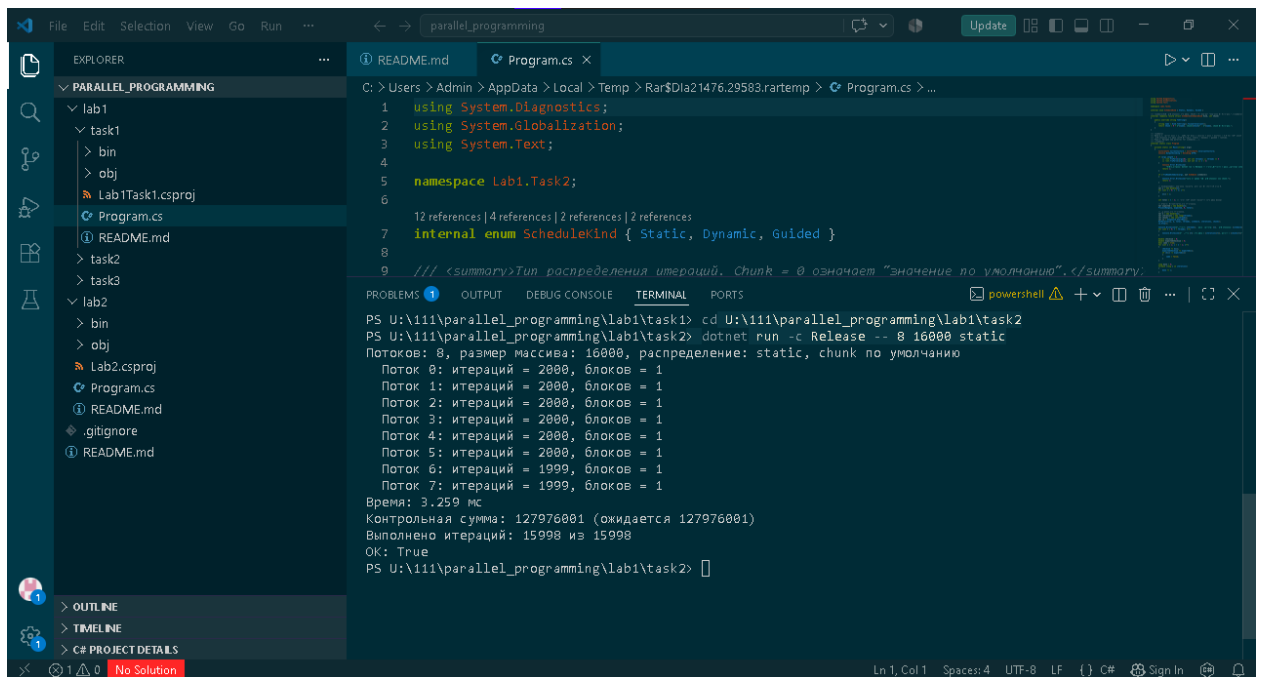


Рис. 2 – запуск с 8 потоками, расписание static

На рисунке 3 показан запуск с расписанием static и блоком 1000: итерации раздаются потокам циклически блоками по 1000 (каждый поток получает несколько блоков). Результат идентичен рисунку 2.

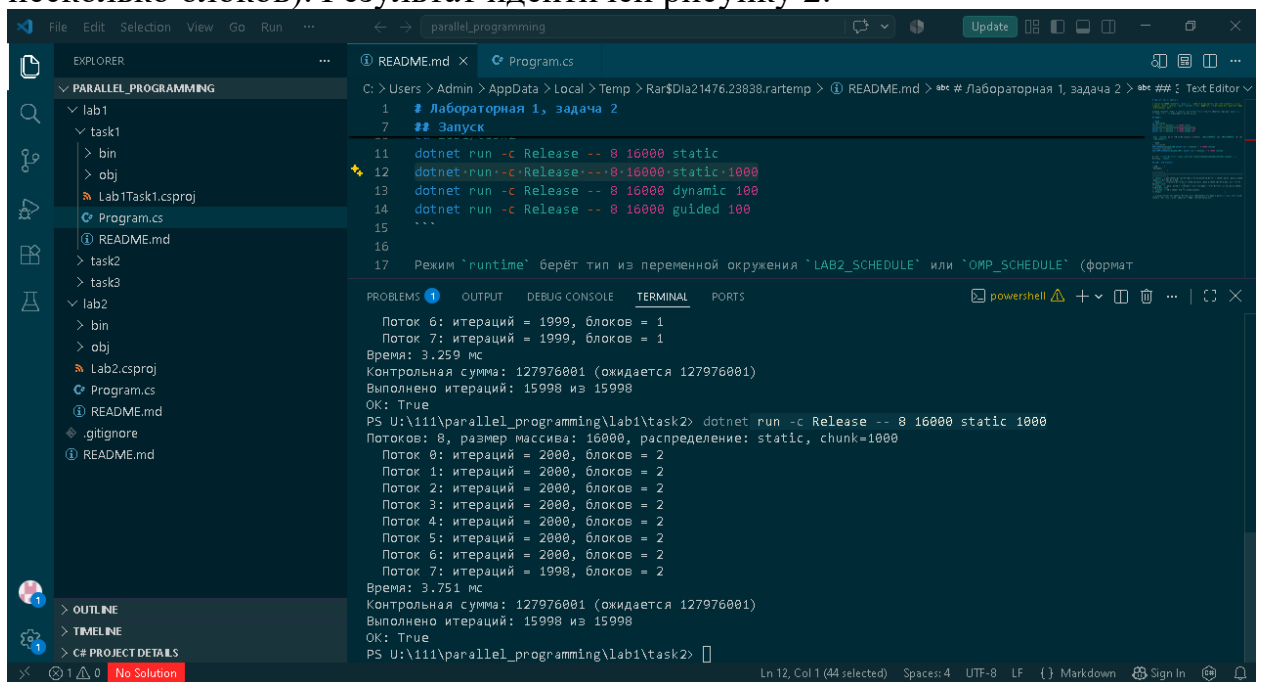


Рис. 3 – запуск с 8 потоками, расписание static с блоком 1000

На рисунке 4 показан запуск с расписанием dynamic и блоком 100: потоки захватывают блоки из общей очереди по мере освобождения, поэтому число блоков у разных потоков может различаться. Все 15998 итераций обработаны, контрольная сумма совпала.

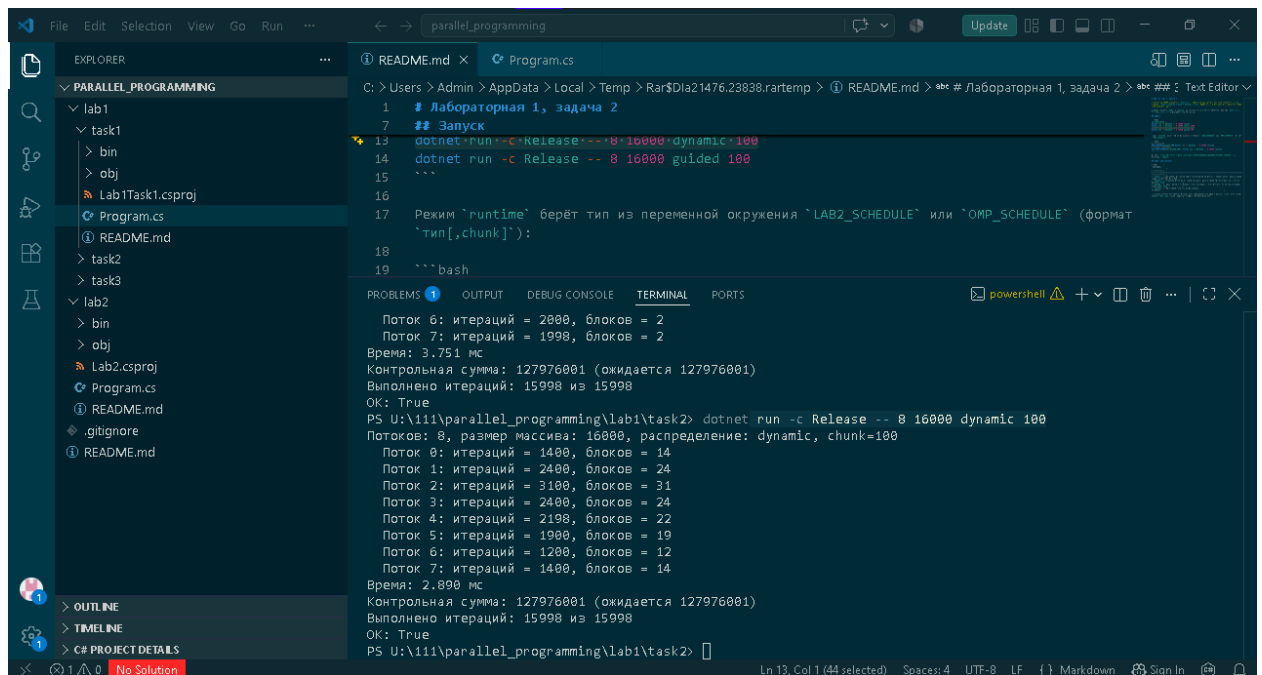


Рис. 4 – запуск с 8 потоками, расписание dynamic с блоком 1000

На рисунке 5 показан запуск с расписанием guided и минимальным блоком 100: размер выдаваемых блоков уменьшается по мере обработки массива, что позволяет балансировать загрузку при меньших накладных расходах. Результат совпадает с предыдущими запусками.

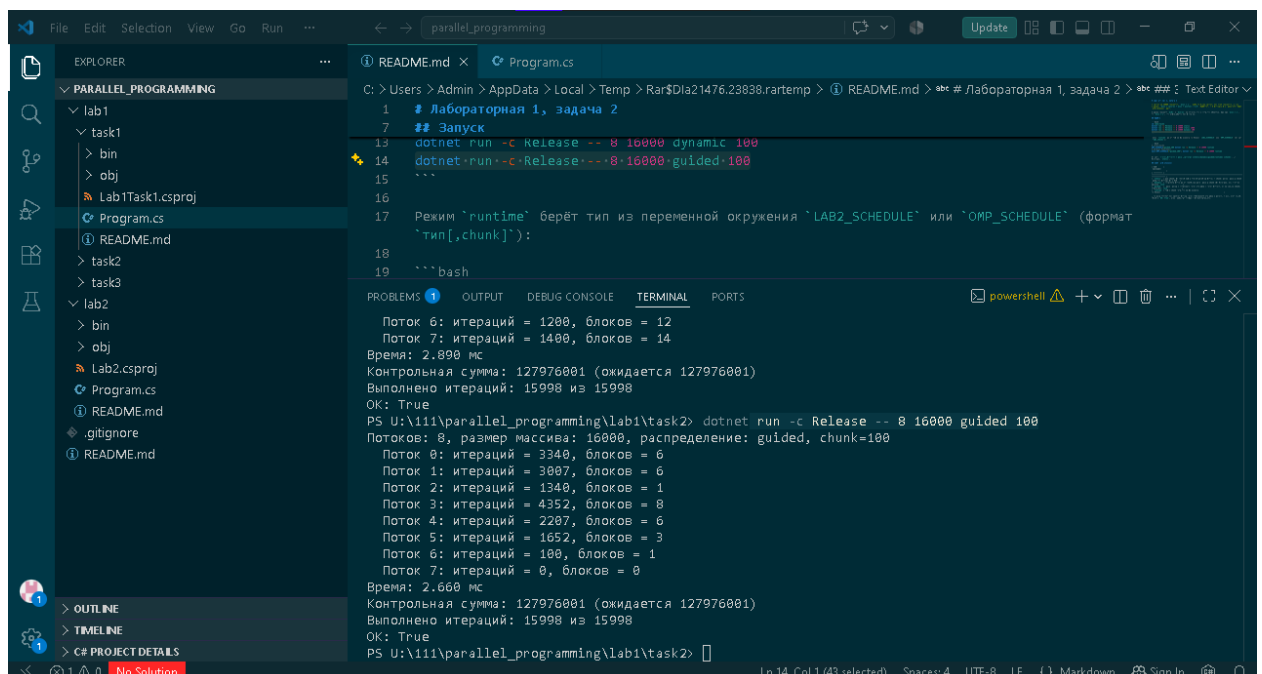


Рис. 5 – запуск с 8 потоками, расписание guided с блоком 1000

На рисунке 6 показан запуск с расписанием runtime: конкретный тип распределения (dynamic,100) взят из переменной окружения

LAB2_SCHEDULE, что соответствует поведению OMP_SCHEDULE в OpenMP.

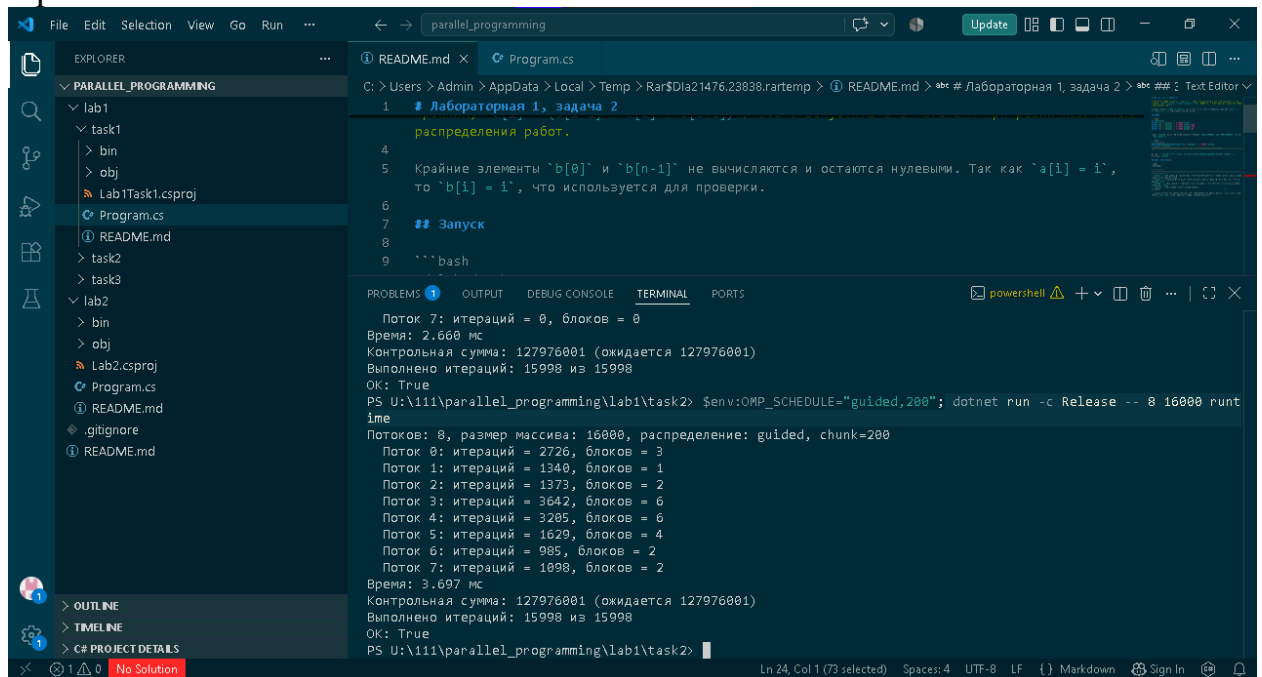


Рис. 6 – запуск с 8 потоками, расписание runtime (dynamic, 100 из окружения)

Задача 3.

Для фиксации порядка вывода необходимы средства синхронизации. В работе реализовано пять способов (разные версии программы выбираются аргументом командной строки): 1) активное ожидание (spinlock) на атомарном счётчике из пакета sync/atomic: поток крутится в цикле, пока счётчик не примет значение его идентификатора; 2) пассивное ожидание на условной переменной sync.Cond: потоки засыпают и пробуждаются сигналом, когда подходит их очередь; 3) цепочка каналов (передача эстафеты): поток i ожидает сигнал от потока $i+1$, печать начинается с потока $N-1$; 4) координатор с массивом каналов и рукопожатием: главный поток выдаёт разрешения на печать в обратном порядке и дожидается подтверждения каждого потока; 5) цепочка sync.WaitGroup: поток i ожидает завершения печати потока $i+1$ через `wgs[i].Wait()`.

На рисунке 7 показана версия программы со способом 1 (активное ожидание на атомарном счётчике). Идентификаторы печатаются строго в порядке 7, 6, 5, 4, 3, 2, 1, 0, что подтверждает корректность способа.

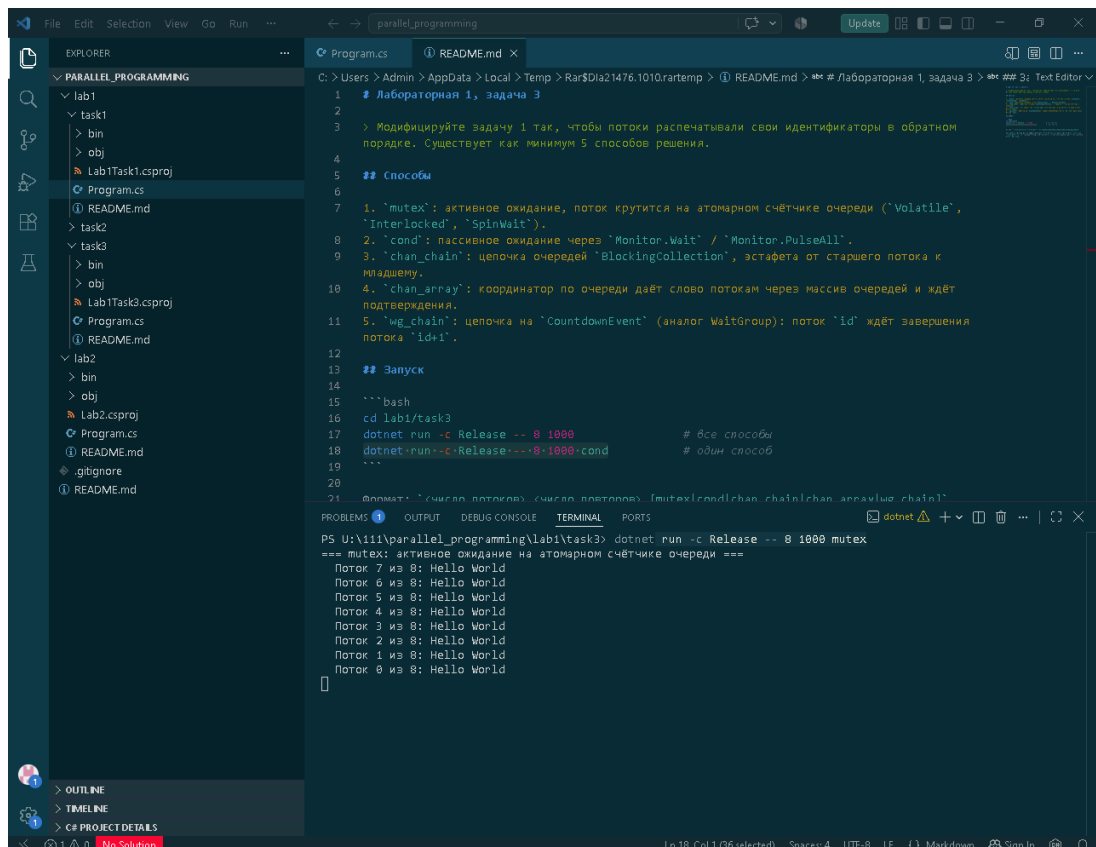


Рис. 7 – способ 1 (активное ожидание, атомарный счётчик)

На рисунке 8 показана версия со способом 2 (условная переменная sync.Cond). Потоки ожидают своей очереди без нагрузки на процессор; порядок вывода также строго обратный.

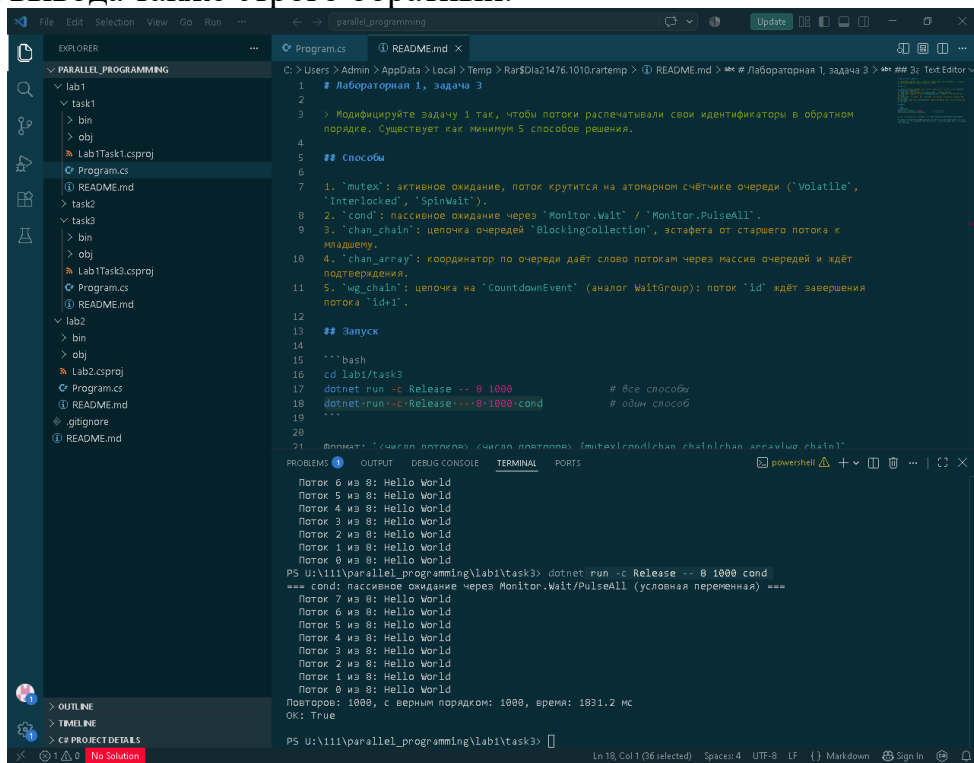


Рис. 8 – способ 2 (Условная переменная sync.Cond)

На рисунке 9 показана версия со способом 3 (цепочка каналов). Право печати передаётся от потока 7 к потоку 0; порядок вывода строго обратный.

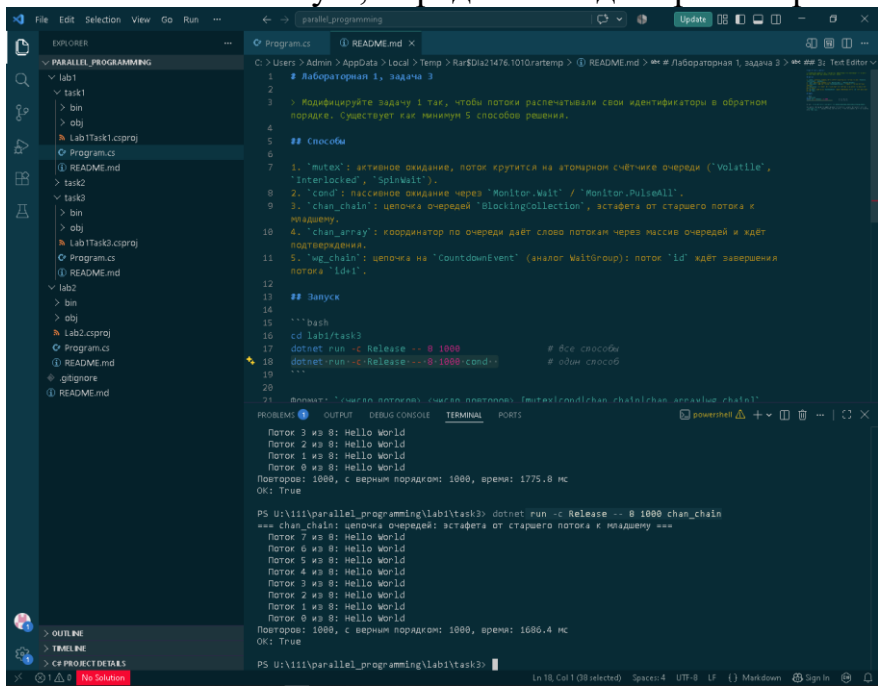


Рис. 9 – способ 3 (цепочка каналов)

На рисунке 10 показана версия со способом 4 (координатор). Главный поток выдаёт разрешения в порядке 7, 6, ..., 0 и дожидается подтверждения каждого потока — наиболее жёсткий контроль порядка

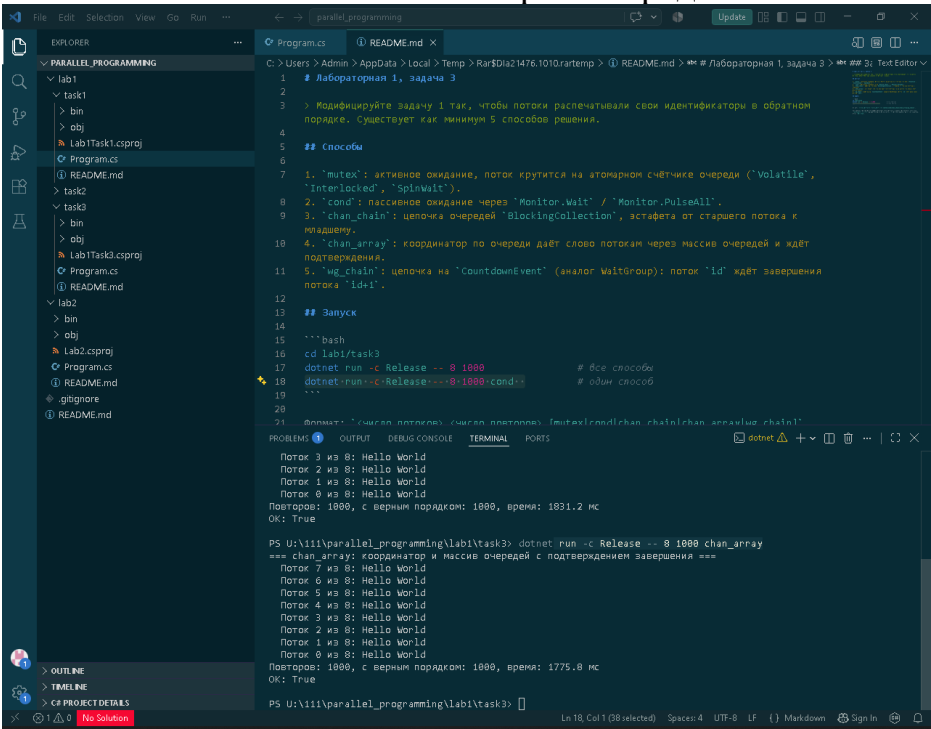


Рис. 10 – способ 4 (Координатор с массивом каналов)

На рисунке 11 показана версия со способом 5 (цепочка sync.WaitGroup).

Каждый поток ждёт завершения потока с большим номером; порядок вывода строго обратный.

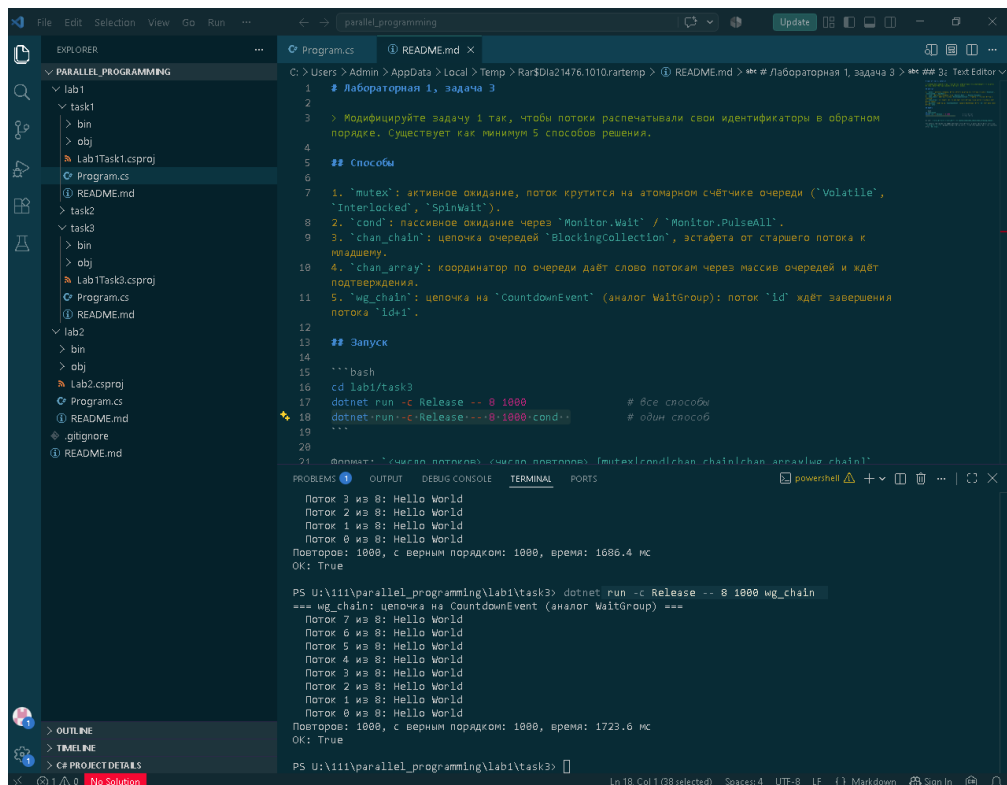


Рис. 10 – способ 5 (цепочка sync. WaitGroup)

Ответы на вопросы, поставленные в задачах

Задача 1: «Всегда ли вывод идентичен? Почему?»

Нет, вывод не всегда идентичен. Потоки выполняются параллельно, и порядок, в котором они достигают оператора вывода, определяется планировщиком, а не программой. Во всех запусках печатается один и тот же набор строк (по одной от каждого из 8 потоков), но последовательность этих строк может меняться. Это подтверждено тремя запусками программы.

Задача 2: вопрос не поставлен В задаче 2 вопрос не ставится; требовалось выполнить запуск с 8 потоками при различных типах распределения работ, что подтверждено рисунками 4–8. Ключевое наблюдение: при независимых итерациях результат (контрольная сумма) одинаков для всех типов расписания, различаются только распределение итераций по потокам и время выполнения.

Задача 3: «Существует как минимум 5 способов решения» Реализовано и продемонстрировано 5 способов фиксации обратного порядка вывода потоков: (1) активное ожидание на атомарном счётчике; (2) условная

переменная `sync.Cond`; (3) цепочка каналов с передачей эстафеты; (4) координатор с массивом каналов и рукопожатием; (5) цепочка `sync.WaitGroup`. Все пять версий показали корректный порядок 7, 6, ..., 0. Аналогом упорядоченного вывода в OpenMP является директива `ordered`. Существуют и другие способы (например, искусственные задержки, зависящие от номера потока), однако они не гарантируют порядок и поэтому в работе не использовались.